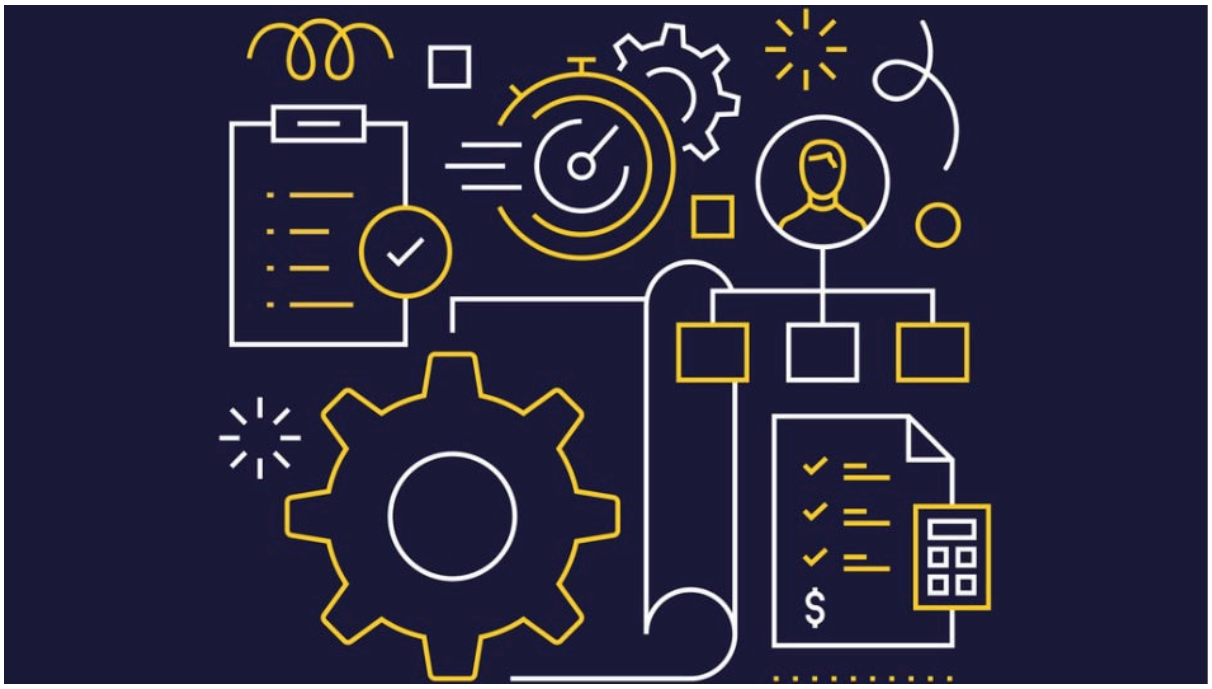


NOTAS DE CLASE
GESTIÓN DE PROYECTOS

{Con ejemplos de programación}

/* ****** **Jaime E. Montoya M.** ****** */

NOTAS DE CLASE
GESTIÓN DE PROYECTOS

{Con ejemplos de programación}

```
/**
* Versión 1.0
* Fecha: 2026, semestre 2
* Licencia software: GNU GPL
* Licencia doc: GNU Free Document License (GNU FDL)
*/


class Author {
      String name = "Jaime E. Montoya M.";

      String profession = "Ingeniero Informático";

      String employment = "Docente y desarrollador";

      String city = "Medellín - Antioquia - Colombia";

      int year = 2026;
}
```

Contenido

Abreviaturas 6
Gestión de Proyectos de Tecnologías de la Información (TI) 8
- Introducción a la Gobernanza de Proyectos TI 8
  - Diferencia entre Gobernanza y Gestión 8
  - Marcos de Referencia Principales 8
    - COBIT (Control Objectives for Information and Related Technologies) 8
    - ITIL (Information Technology Infrastructure Library) 9

Ciclo de Vida del Proyecto: Predictivo vs. Adaptativo 12
- Enfoque Predictivo (Cascada / Tradicional) 12
- Enfoque Adaptativo (Ágil / Iterativo e Incremental) 12
- Cuadro Comparativo: Predictivo vs. Adaptativo 12

Datos complementarios: acerca de los cargos de gerentes de proyectos, líderes técnicos y roles de scrum en Colombia 14
- Salarios promedios y aproximados en Colombia 14
  - Rangos según la Experiencia 14
  - Factores que afectan el sueldo 14
- La Estructura Ideal (Empresas Medianas y Grandes) 15
  - La Realidad en Empresas Pequeñas o Startups 15
    - El "PM + Scrum Master" (El rol híbrido) 15
    - El "PM + Líder Técnico" (Poco recomendado) 16
  - Resumen Organizacional 16
  - Fuentes 16
- Cantidad de proyectos a cargo 17
  - Gerente de Proyectos de TI (PM) 17
  - Scrum Master 17
  - Líder Técnico (Tech Lead) 17
  - Resumen de Capacidad Organizacional 18
  - Fuentes 18
- Equipos grandes y múltiples proyectos 19
  - Director de la PMO (Project Management Office) / Gerente de Portafolio 19
  - Director de Tecnología (VP of Engineering / IT Director) 19
  - Agile Coach Principal / Agile Delivery Manager 19
  - La Cúspide: El CIO o CTO 19
  - ¿Cómo se ve la estructura en resumen? 20

Fuentes 20
Estándares Globales: PMBOK y PRINCE2 21
PMBOK (Project Management Body of Knowledge - PMI) 21
Guía del PMBOK y su Aplicación en Proyectos de TI 21
Contexto General del PMBOK 21
¿Para qué sirve la Guía del PMBOK? 21
Importancia Actual en la Era Digital 22
Aplicación Práctica en Proyectos de TI 22
Planificación e Incertidumbre (Gestión del Alcance y Riesgos TI) 22
Enfoque de Desarrollo y Ciclo de Vida 22
Entrega de Valor Continua 23
Gestión de Interesados (Stakeholders) 23
Datos de Valor y Recomendaciones 23
PRINCE2 (PRojects IN Controlled Environments) 23
GUÍA PRÁCTICA DE PRINCE2 (Projects IN Controlled Environments) 23
Contexto General de PRINCE2 24
¿Para qué sirve PRINCE2? 24
La Estructura de PRINCE2 24
Los 7 Principios (El 'Por Qué') 24
Las 7 Prácticas (El 'Qué') 25
Los 7 Procesos (El 'Cómo') 25
Importancia Actual de PRINCE2 25
Aplicación de PRINCE2 en Proyectos de TI 26
Gestión de Despliegues de Infraestructura y Migraciones en la Nube 26
Control estricto de Tolerancias contra el 'Scope Creep' Tecnológico 26
Datos Adicionales y Comparativa de Mercado 27
Cuadro Comparativo: PMBOK vs. PRINCE2 27
Estudios de Viabilidad Técnica, Operativa y Financiera 29
Recursos para un proyecto 29
Humanos 29
Técnicos / Tecnológicos 29
Económicos / Financieros 29
Operativos 30
Estudio de factibilidad / viabilidad de un proyecto 30
Importancia de la factibilidad 30
Factibilidad humana 31
Factibilidad técnica/tecnológica 31
Factibilidad económica 31
Factibilidad operacional 31

- Tipos de Viabilidad en gestión de proyectos 32
- Framework XP (Extreme Programming) en Proyectos de TI 33
  - Introducción a Extreme Programming (XP) 33
  - Los 5 Valores Fundamentales de XP 33
  - Las 12 Prácticas Clave de XP 33
  - Cómo Aplicar XP en Proyectos de TI 34
    - Fase 1: Preparación y Descubrimiento 34
    - Fase 2: Planificación del Release (Lanzamiento) 35
    - Fase 3: Ciclos de Iteración (Semanales o Quincenales) 35
    - Fase 4: Integración y Despliegue Continuo 35
    - Fase 5: Cierre de Iteración y Retroalimentación 35
  - ¿Por qué evitar la división tradicional de Caja Blanca / Caja Negra en TDD? 36
  - Red-Green-Refactor 36
  - Las 3 Reglas del TDD (Por Uncle Bob) 37
  - Los Beneficios Reales (Los Trade-offs) 37
  - Un Ejemplo Mental Rápido 37
  - Integración Continua (CI) con GitHub Actions 48
  - Fuentes Consultadas y Lecturas Recomendadas 54
- Indicadores Financieros de Evaluación de Proyectos 55
  - ROI (Retorno de la Inversión / Return on Investment) 55
  - VAN (Valor Actual Neto) / NPV (Net Present Value) 55
    - Equivalencia periódica (Consistencia temporal) 56
    - ¿De dónde sale el valor de la tasa r? 56
  - TIR (Tasa Interna de Retorno) / IRR (Internal Rate of Return) 57
- Planificación Macro, Gestión de Stakeholders y Líneas Base 60
  - Planificación Macro 60
  - Gestión de Interesados (Stakeholders) 60
  - Triángulo de Restricciones y Líneas Base (Alcance, Tiempo, Costo) 60
- Gestión Tradicional de Riesgos y Matriz de Riesgos 62
  - Evaluación Cualitativa: Matriz de Probabilidad e Impacto 62
  - Estrategias de Respuesta a Riesgos Negativos 62
- Ética Profesional en Proyectos de TI 63
  - Código de Ética y Conducta Profesional del PMI 63
  - Dilemas Éticos Frecuentes en TI 63
- Bibliografía y Lecturas Complementarias 64

Abreviaturas

BD: Base de Datos

COBIT: Control Objectives for Information and Related Technologies

CRM: Customer Relationship Management (Gestión de Relacionamiento con el Cliente)

DD: Diccionario de Datos

DFD: Diagrama de Flujo de Datos

ERP: Enterprise Resource Planning (Planeación de Recursos Empresariales)

ISO: International Organization for Standardization (Organización Internacional de Estandarización)

ITIL: Information Technology Infrastructure Library

PM: Project Management (Gerente del Proyecto)

PMBOK: Project Management Body of Knowledge

PMI: Project Management Institute

PMO: Project Management Office (Oficina de Gestión del Proyecto)

PO: Product Owner (Propietario del Producto)

PRINCE2: PRojects IN Controlled Environments

ROI: Return On Investment (Retorno de la Inversión)

SGE: Sistema de Gestión Empresarial

SGSI: Sistema de Gestión de Seguridad de la Información

SI: Sistema de Información

SO: Sistema Operativo

TGS: Teoría General de Sistemas

TIC: Tecnologías de la Información y la Comunicación

TIR: Tasa Interna de Retorno / (**IRR**: Internal Rate of Return)

UML: Unified Modeling Language (Lenguaje de Modelado Unificado)

VAN: Valor Actual Neto (**NPV**: Net Present Value)

Gestión de Proyectos de Tecnologías de la Información (TI)

Esta guía está diseñada como material docente para acompañar el aprendizaje de los estudiantes en los fundamentos de la gobernanza, los ciclos de vida del desarrollo de software y proyectos TI, los estándares internacionales más reconocidos, la evaluación financiera y técnica de proyectos, la planificación macro y la gestión de riesgos y ética.

Introducción a la Gobernanza de Proyectos TI

La **Gobernanza de Proyectos TI** se define como el marco de trabajo, principios, políticas y estructuras organizacionales mediante los cuales se asegura que las inversiones en tecnologías de la información estén alineadas con la estrategia del negocio, entreguen valor esperado y gestionen de manera adecuada los riesgos asociados.

Diferencia entre Gobernanza y Gestión

Es fundamental no confundir la *Gobernanza* con la *Gestión (Management)*:

- **Gobernanza (Evaluar, Dirigir y Monitorear):** Define *qué* se debe hacer, quién toma las decisiones, cuáles son las prioridades estratégicas y cómo se evalúa el éxito global. Corresponde habitualmente a la alta dirección, comités directivos y juntas gerenciales.
- **Gestión (Planificar, Construir, Ejecutar y Monitorear):** Se encarga de *cómo* se ejecutan las actividades operativas y proyectos en el día a día para alcanzar los objetivos fijados por la gobernanza. Corresponde a directores de proyectos, líderes técnicos y equipos de trabajo.

Marcos de Referencia Principales

- **COBIT (*Control Objectives for Information and Related Technologie*s):** Marco global enfocado en la gobernanza y gestión integral de la TI empresarial.
- **ITIL (*Information Technology Infrastructure Library*):** Centrado en la gestión de servicios de TI para alinearlos con las necesidades del negocio.

COBIT (*Control Objectives for Information and Related Technologies*)

Es un marco de referencia creado por ISACA para la gobernanza y gestión de la tecnología y la información en las empresas. COBIT significa **Objetivos de Control para la Información y Tecnologías Relacionadas**; ayuda a conectar los objetivos del negocio con las acciones del departamento de tecnología. No se enfoca solo en la parte técnica, sino en toda la empresa.

Gobierno vs. Gestión
COBIT separa dos conceptos clave:

Gobierno: Evalúa las necesidades de los dueños de la empresa, fija prioridades y revisa los resultados, además fija dirección y monitorea (responsabilidad de la alta dirección). Lo hace la alta dirección.

Gestión: Planifica, construye, ejecuta y monitorea las tareas diarias siguiendo las reglas del gobierno. Lo hacen los equipos operativos.

Estructura de COBIT 2019
La versión más reciente organiza el trabajo de forma lógica:

- Cuenta con 40 objetivos Core principales de gobierno y gestión. Se dividen en 5 dominios principales:
    - EDM (Evaluar, Dirigir y Monitorear): 5 objetivos de gobierno.
    - APO (Alinear, Planificar y Organizar): 14 objetivos de gestión
    - BAI (Construir, Adquirir e Implementar): 11 objetivos de gestión.
    - DSS (Entregar, Servir y Soportar): 6 objetivos de gestión.
    - MEA (Monitorear, Evaluar y Valorar): 4 objetivos de gestión.
- 7 Componentes del Sistema: Procesos, estructuras organizativas, flujos de información, cultura/ética, personas/competencias, políticas/procedimientos, y servicios/infraestructura.
- Factores de diseño:Permiten adaptar el sistema de gobierno a la estrategia, tamaño, perfil de riesgo y necesidades de cumplimiento de cada organización.
- Incluye guías para diseñar un sistema a la medida de cada empresa.
- Usa métricas para medir el éxito y el nivel de capacidad de los procesos.

¿Para qué sirve?
Crear valor: Asegura que las inversiones en tecnología ayuden a ganar dinero o mejorar el servicio.

Reducir riesgos: Controla los peligros informáticos y protege los activos y datos de la organización.

Cumplir normas: Facilita adaptarse a leyes de seguridad y auditorías externas.

ITIL (*Information Technology Infrastructure Library*)

Es el marco de referencia de buenas prácticas más utilizado en el mundo para la gestión de servicios de tecnologías de la información (ITSM). Su objetivo principal es ayudar a las organizaciones a alinear sus servicios tecnológicos con las necesidades reales del negocio, mejorando la eficiencia operativa y entregando valor continuo.

ITIL, la **Biblioteca de Infraestructura de Tecnologías de Información**, permite la alineación con el negocio, ya que ayuda a que los departamentos de tecnología no trabajen aislados, sino enfocados en aportar valor real a los objetivos de la empresa. Facilita la eficiencia operativa

estandarizando procesos como la gestión de incidencias, problemas y cambios para reducir errores y tiempos de respuesta, además que propende por la mejora continua, promoviendo la evolución constante de los servicios, basándose en la retroalimentación y las necesidades del cliente.

Características

- **Sistema de Valor del Servicio (SVS)**: Describe cómo todos los componentes y actividades de una organización trabajan juntos para co-crear valor a partir de la demanda de los clientes.
- **Prácticas de gestión**: Conjunto de recursos organizativos diseñados para realizar trabajo o cumplir un objetivo (como la mesa de servicio o *service desk*).
- **Principios rectores**: Guías de acción flexibles como "enfocarse en el valor", "mantener la simplicidad" o "avanzar de forma iterativa".

Evolución: ITIL v3 vs. ITIL 4

La principal diferencia es el enfoque operativo y la adaptación a entornos modernos de trabajo:

- **ITIL v3 (Ciclo de Vida)**: Se estructuraba en 5 etapas secuenciales (Estrategia, Diseño, Transición, Operación y Mejora Continua del Servicio). Era criticado por ser rígido y propiciar silos.
- **ITIL 4 (Ecosistema de Valor)**: Sustituye el ciclo de vida por el Sistema de Valor del Servicio (SVS). Integra filosofías ágiles modernas como DevOps, Lean y Agile, permitiendo flujos de trabajo más rápidos y flexibles.

Las 4 Dimensiones de la Gestión de Servicios
Para garantizar un enfoque holístico (integral), ITIL 4 establece que cada servicio debe analizarse a través de cuatro dimensiones. Si una falla, el servicio no entrega el valor esperado

- **Organizaciones y personas**: Cultura corporativa, competencias, roles y canales de comunicación efectivos.
- **Información y tecnología**: El conocimiento necesario, las bases de datos y las herramientas tecnológicas (como softwares ITSM).
- **Socios y proveedores**: Relaciones con terceros, contratos, integradores de servicios y alianzas estratégicas.
- **Flujos de valor y procesos**: La secuencia exacta de actividades que se realizan para crear y entregar un producto o servicio de forma eficiente.

Esquema de Certificación ITIL 4
El camino de certificación oficial está diseñado para profesionales que buscan validar sus conocimientos en gestión de TI:

- **ITIL Foundation**: El nivel inicial. Certifica que comprendes la terminología, los conceptos clave y la estructura general de ITIL 4.
- **ITIL Managing Professional (MP)**: Dirigido a profesionales técnicos y operativos. Evalúa cómo dirigir proyectos y gestionar flujos de trabajo tecnológicos.
- **ITIL Strategic Leader (SL)**: Dirigido a líderes y gerentes. Se enfoca en cómo la tecnología guía y da forma a la estrategia del negocio.

- **ITIL Master**: El nivel máximo. Requiere demostrar experiencia práctica aplicando el marco de referencia en situaciones reales de negocio.

Aplicación Práctica: Gestión de Incidencias
Para entender cómo funciona en el día a día, la Gestión de Incidencias es una de las prácticas más comunes. Su objetivo es restaurar la operación normal del servicio lo antes posible. Su flujo típico es:

- **Registro**: El usuario reporta una falla (ej. "no funciona el correo") a través del Service Desk y se categoriza.
- **Priorización**: Se asigna un nivel de urgencia según el impacto que cause al negocio.
- Diagnóstico y Resolución: El equipo técnico identifica la solución o aplica un *workaround* (solución temporal).
- **Cierre**: Se verifica con el usuario que el servicio funciona correctamente y se documenta la lección aprendida.

Ciclo de Vida del Proyecto: Predictivo vs. Adaptativo

El ciclo de vida de un proyecto describe las fases desde su inicio hasta su cierre. Dependiendo del grado de incertidumbre, estabilidad de requerimientos y velocidad de entrega requerida, se seleccionan enfoques tradicionales o ágiles.

Enfoque Predictivo (Cascada / Tradicional)

En el enfoque predictivo, el alcance, costo y tiempo se determinan en las primeras fases del proyecto. Se basa en una planificación detallada por adelantado y sigue una secuencia rígida (Análisis → Diseño → Desarrollo → Pruebas → Despliegue).

- **Ventajas:** Alta previsibilidad, documentación clara y facilidad de control presupuestal inicial.
- **Desventajas:** Poca flexibilidad ante cambios, el cliente solo ve el producto final en etapas muy tardías y alto riesgo si los requerimientos cambian.

Enfoque Adaptativo (Ágil / Iterativo e Incremental)

En el enfoque adaptativo, el alcance se refina de forma continua a lo largo del proyecto a través de iteraciones cortas (sprints de 1 a 4 semanas). Se prioriza la entrega de valor temprano y continuo.

- **Ventajas:** Alta capacidad de respuesta al cambio, retroalimentación temprana del usuario y mitigación continua de riesgos.
- **Desventajas:** Menor previsibilidad del alcance total inicial y requiere un compromiso constante del cliente/Product Owner.

Cuadro Comparativo: Predictivo vs. Adaptativo

Criterio	Enfoque Predictivo (Tradicional)	Enfoque Adaptativo (Ágil)
Requerimientos	Fijos y definidos al inicio del proyecto.	Dinámicos, cambian y se refinan en el backlog.
Planificación	Plan detallado a largo plazo (Diagrama de Gantt).	Planificación iterativa (Sprint Planning / Release Planning).
Gestión del Cambio	Restringida; requiere un proceso formal de control de	Bienvenida; el cambio es parte natural del proceso.

Criterio	Enfoque Predictivo (Tradicional)	Enfoque Adaptativo (Ágil)
	cambios.	
Entrega de Valor	Al final del ciclo del proyecto.	Incremental al final de cada iteración.
Participación del Cliente	Puntos de control específicos (hitos) e hito final.	Colaboración continua y diaria.

Datos complementarios: acerca de los cargos de gerentes de proyectos, líderes técnicos y roles de scrum en Colombia

Salarios promedios y aproximados en Colombia

El salario promedio de un gerente de proyectos de TI (*IT Project Manager*) en Colombia oscila **entre $6.000.000 y $10.000.000 COP al mes** (aproximadamente entre $1.400 y $2.500 USD), variando según la experiencia, el tamaño de la compañía y si se trabaja para empresas locales o de forma remota para el exterior.[1] [2] [3] [1][2]

Rangos según la Experiencia

- **Junior (menos de 2 años):** $3.500.000 a $5.500.000 COP mensuales.
- **Mid-Level (de 2 a 5 años):** $6.000.000 a $9.000.000 COP mensuales.
- **Senior (más de 5 años):** $10.000.000 a $18.000.000+ COP mensuales (pudiendo superar estos valores si el pago es en divisas).

Factores que afectan el sueldo

- **Modalidad de contratación:** Trabajar en inglés para empresas internacionales en remoto suele duplicar o triplicar el ingreso local en pesos.
- **Certificaciones:** Contar con credenciales reconocidas como PMP, Scrum Master (PSM/CSM) o ITIL incrementa significativamente la oferta salarial.
- **Ubicación de la empresa:** Ciudades como Bogotá y Medellín concentran las ofertas con los presupuestos tecnológicos más altos del país.

En la teoría de la gestión de proyectos, **el Gerente de Proyectos (PM) y el Scrum Master (o Líder Técnico) son roles distintos con enfoques diferentes**. Sin embargo, la realidad organizacional varía drásticamente según el tamaño de la empresa. [1]

---

[1] Los mejores salarios de Colombia - El salario de un gerente en Colombia - Séligo
[2] Salario de un Project Manager en Colombia 2026 — Compara y Calcula | Talently
[3] Sueldo: Consultor Junior en Bogota, Colombia 2026 | Glassdoor

A continuación, te detallo cómo se dividen las responsabilidades y cómo cambia esta estructura según la escala de la organización:

La Estructura Ideal (Empresas Medianas y Grandes)

En organizaciones con metodologías ágiles bien estructuradas, los roles están separados para evitar conflictos de interés y sobrecarga:

- **Gerente de Proyectos de TI (Enfoque Estratégico/Comercial):** Se enfoca en el *por qué* y el *cuándo*. Gestiona el presupuesto, los contratos, los proveedores, las expectativas del cliente y los riesgos del negocio. Mira el panorama completo del proyecto. [2, 3, 4]
- **Scrum Master (Enfoque en Procesos/Equipo):** Se enfoca en el *cómo*. Es un facilitador que ayuda al equipo a aplicar Scrum, elimina obstáculos internos y protege al equipo de interrupciones externas. No gestiona presupuestos ni tiene autoridad jerárquica. [5, 6, 7, 8, 9]
- **Líder Técnico (Tech Lead - Enfoque Técnico):** Se enfoca en el *qué* y la *calidad de ingeniería*. Define la arquitectura del software, guía técnicamente a los desarrolladores y asegura que el código sea de alta calidad.

La dinámica de comunicación: El Gerente de Proyectos está en constante comunicación con el Scrum Master y el Líder Técnico. El PM no microgestiona el día a día del desarrollo; en su lugar, recibe actualizaciones de ellos sobre el progreso, bloqueos técnicos o retrasos que afecten el presupuesto o las fechas de entrega con el cliente.

La Realidad en Empresas Pequeñas o Startups

En empresas pequeñas **es muy común y viable combinar roles**, pero esto conlleva riesgos organizacionales específicos: [10]

El "PM + Scrum Master" (El rol híbrido)

- **Cómo funciona:** Una sola persona gestiona el presupuesto y el cliente (PM), mientras facilita las reuniones diarias y remueve bloqueos del equipo de desarrollo (Scrum Master). [11]
- **El riesgo:** Existe un conflicto de interés natural. El PM suele presionar por cumplir fechas y presupuestos, mientras que el Scrum Master debe proteger el ritmo de trabajo saludable del equipo. Al fusionarlos, el rol de PM casi siempre termina absorbiendo al Scrum Master, priorizando la entrega rápida sobre la calidad del proceso ágil. [12]

El "PM + Líder Técnico" (Poco recomendado)

- **Cómo funciona:** Un ingeniero senior gestiona la arquitectura del software y, a la vez, se encarga del presupuesto, reuniones con clientes y reportes financieros.
- **El riesgo:** Es una de las combinaciones más difíciles. La gestión administrativa consume tanto tiempo que el líder pierde el enfoque técnico, o viceversa, descuidando al cliente por resolver un problema en el código.

Resumen Organizacional

- **En empresas grandes:** El Gerente de TI es puramente estratégico/administrativo y coordina **con** los líderes técnicos y Scrum Masters.
- **En empresas pequeñas:** El Gerente de TI suele ser un "todólogo" que asume la estrategia y la facilitación del equipo (PM + Scrum Master), dejando la parte técnica a un desarrollador senior (Líder Técnico) para no colapsar.

Fuentes

[1] https://apmg-international.com

[2] https://nuclio.school

[3] https://kinetic-corp.com

[4] https://www.adecco.com

[5] https://asana.com

[6] https://dharmacon.net

[7] https://teachingagile.com

[8] https://www.iebschool.com

[9] https://www.postgradoutp.edu.pe

[10] https://kinetic-corp.com

[11] https://www.freelancermap.com

[12] https://www.advisedskills.com

Cantidad de proyectos a cargo

La cantidad de proyectos que cada rol puede gestionar al tiempo depende de la complejidad de los desarrollos, pero existen límites estándar en la industria para garantizar la eficiencia y evitar el agotamiento del equipo. [1]

Aquí te detallo la capacidad máxima recomendada para cada perfil:

Gerente de Proyectos de TI (PM)

- **Capacidad óptima: 2 a 4 proyectos en paralelo**.
- **De qué depende:** Un PM trabaja a nivel estratégico (presupuestos, clientes y contratos). Si los proyectos son pequeños o están en fase de mantenimiento, puede subir a **5 o 6**. Sin embargo, si son proyectos grandes con muchos interesados (*stakeholders*), gestionar más de 3 destruye la capacidad de analizar riesgos a tiempo. [2, 3]

Scrum Master

- **Capacidad óptima: 1 a 2 equipos (células) de desarrollo**.
- **De qué depende:** En la metodología Scrum, este rol no cuenta "proyectos", sino **equipos**. Un equipo ideal tiene entre 3 y 9 miembros. Un Scrum Master dedicado puede atender un máximo de **2 equipos en paralelo** (lo que podría equivaler a 2 proyectos distintos). Intentar atender 3 o más equipos convierte al Scrum Master en un simple "programador de reuniones", perdiendo el enfoque en mejorar la productividad y resolver bloqueos reales. [4, 5, 6, 7, 8]

Líder Técnico (Tech Lead)

- **Capacidad óptima: 1 solo proyecto a la vez** (máximo 2 si son muy pequeños).
- **De qué depende:** El líder técnico debe diseñar la arquitectura, revisar el código del equipo y guiar el desarrollo diariamente. Debido a la alta carga cognitiva y técnica, **el multi-tasking es crítico para este rol**. Si un líder técnico se divide en más de un proyecto complejo, se convierte en un cuello de botella para ambos equipos, retrasando las decisiones arquitectónicas y bajando la calidad del software. [9, 10, 11, 12]

Resumen de Capacidad Organizacional

Rol	Proyectos / Equipos simultáneos	Enfoque principal
Gerente de Proyectos	2 - 4 proyectos	Negocio, presupuesto y fechas
Scrum Master	1 - 2 equipos	Procesos, agilidad y bloqueos
Líder Técnico	1 proyecto	Arquitectura y calidad de código

Fuentes

[1] https://www.larepublica.net

[2] https://thedigitalprojectmanager.com

[3] https://www.jobsprout.ai

[4] https://echometerapp.com

[5] https://clickup.com

[6] https://abstracta.us

[7] https://www.scrumio.com

[8] https://www.reddit.com

[9] https://clickup.com

[10] https://edgarfernandezr.substack.com

[11] https://www.cesarcancino.com

[12] https://asana.com

Equipos grandes y múltiples proyectos

En empresas medianas y grandes, la proliferación de proyectos y equipos exige una estructura jerárquica clara. Por encima de los gerentes de proyectos de TI individuales, existen roles directivos que unifican la estrategia y centralizan el control.

Esta es la escala jerárquica habitual en la industria, de menor a mayor rango:

Director de la PMO (Project Management Office) / Gerente de Portafolio

Es el jefe directo de los Gerentes de Proyectos (PMs).

- **Su función:** No gestiona proyectos individuales. Coordina el **portafolio** completo de proyectos de la empresa. Asegura que todos los PMs sigan las mismas metodologías, aprueba los presupuestos macro y redistribuye los recursos si un proyecto crítico está en peligro.

Director de Tecnología (VP of Engineering / IT Director)

Es el jefe de los Líderes Técnicos y, a menudo, trabaja a la par con el Director de la PMO. [1]

- **Su función:** Define la estrategia tecnológica a largo plazo de la compañía (por ejemplo, migrar todos los sistemas a la nube o cambiar el lenguaje de programación principal). Se asegura de que la infraestructura soporte el crecimiento del negocio.

Agile Coach Principal / Agile Delivery Manager

Es el mentor y supervisor de los Scrum Masters.

- **Su función:** En empresas grandes con docenas de células ágiles, este rol escala la metodología (usando frameworks como SAFe o Less) para que los diferentes equipos de Scrum no trabajen como islas y se coordinen entre sí.

La Cúspide: El CIO o CTO

En la cima de toda el área de TI de una gran empresa se encuentra un ejecutivo C-Level, que reporta directamente al Gerente General (CEO): [2]

- **CIO (Chief Information Officer):** Enfocado en los sistemas internos, la eficiencia operativa, la ciberseguridad y las herramientas tecnológicas que usa la empresa para operar.
- **CTO (Chief Technology Officer):** Enfocado en el producto tecnológico que la empresa vende hacia afuera, la innovación y la arquitectura técnica de cara al cliente.

¿Cómo se ve la estructura en resumen?

Los **Líderes Técnicos** reportan a un Director de Ingeniería. Los **Scrum Masters** se coordinan con un Agile Coach. Los **Gerentes de Proyectos** reportan al Director de la PMO. Finalmente, todos estos directores responden ante el **CTO o CIO** de la compañía. [3]

Fuentes

[1] https://blog.talentoit.org

[2] https://ecosistemastartup.com

[3] https://echometerapp.com

Estándares Globales: PMBOK y PRINCE2

Para la gestión estructurada de proyectos, existen dos estándares ampliamente reconocidos a nivel mundial:

PMBOK (Project Management Body of Knowledge - PMI)

Publicado por el **Project Management Institute (PMI)**, el **PMBOK** es una guía de buenas prácticas que describe conceptos generales, dominios de desempeño y principios de dirección de proyectos. Proporciona un marco amplio y flexible aplicable a cualquier tipo de proyecto. En su versión 7 (PMBOK 7), se centra en 12 principios y 8 dominios de desempeño, integrando explícitamente enfoques predictivos, híbridos y adaptativos.

Guía del PMBOK y su Aplicación en Proyectos de TI

Contexto, importancia y adopción ágil en entornos tecnológicos

Contexto General del PMBOK

La Guía del PMBOK (Project Management Body of Knowledge) es el estándar global de facto para la gestión de proyectos, desarrollado y actualizado continuamente por el Project Management Institute (PMI). Más que un método rígido, consiste en una compilación de buenas prácticas, vocabulario común y lineamientos estructurados que guían a los directores de proyectos a nivel internacional.

> *Evolución Clave: La transición de la 6ª a la 7ª edición marcó un hito histórico. Se pasó de un enfoque estrictamente basado en Procesos y Áreas de Conocimiento (Entradas, Herramientas, Salidas) hacia un modelo centrado en Principios de Entrega de Proyectos y Dominios de Desempeño. Este cambio responde directamente a la necesidad de adaptabilidad en entornos volátiles y digitales.*

¿Para qué sirve la Guía del PMBOK?

El propósito fundamental del PMBOK es profesionalizar y estandarizar la ejecución de iniciativas corporativas mediante:

- **Estandarización del lenguaje:** Unificar la terminología utilizada por equipos, proveedores y clientes en todo el mundo.
- **Flexibilidad metodológica:** Provee una caja de herramientas de modelos, métodos y artefactos adaptables al tamaño y complejidad de cualquier proyecto.
- **Orientación al valor:** Enfoca la gestión en la entrega de valor real al negocio, no solo en cumplir plazos o presupuestos de forma aislada.
- **Control de riesgos y gobernanza:** Ofrece pautas rigurosas para identificar, mitigar y monitorear desviaciones financieras o de cronograma.

Importancia Actual en la Era Digital

En el mercado contemporáneo, marcado por la incertidumbre y la velocidad del cambio tecnológico, el PMBOK mantiene una relevancia crítica debido a tres factores esenciales:

Hibridación de enfoques: Hoy en día los proyectos no admiten un enfoque único. La guía actual enseña cómo calibrar el nivel de gobernanza según el ciclo de vida del proyecto (predictivo, híbrido o ágil).

Enfoque en Outcomes sobre Outputs: Ayuda a los líderes a enfocarse en los resultados del proyecto (beneficios obtenidos) en lugar de obsesionarse únicamente con los productos entregables físicos o de software.

Desarrollo de talento integral: Las empresas demandan directores de proyecto que entiendan la estrategia del negocio, manejen habilidades blandas (liderazgo, empatía) y dominen la técnica. El PMBOK soporta de forma integral este Triángulo de Talento del PMI.

Aplicación Práctica en Proyectos de TI

Los proyectos de Tecnologías de la Información (TI) —como desarrollo de software, migración a la nube o implementación de ciberseguridad— sufren de cambios de alcance constantes. La aplicación moderna del PMBOK en TI se despliega a través de sus Dominios de Desempeño:

Planificación e Incertidumbre (Gestión del Alcance y Riesgos TI)

En TI, los requerimientos iniciales rara vez son los finales. El PMBOK promueve la 'elaboración progresiva'. Se definen los riesgos de infraestructura, deuda técnica e integraciones desde el inicio, pero el alcance detallado se va adaptando mediante iteraciones.

Enfoque de Desarrollo y Ciclo de Vida

Permite estructurar proyectos híbridos. Por ejemplo, en la implementación de un sistema ERP core corporativo: la infraestructura en la nube y la compra de licencias se gestionan con un enfoque predictivo (cascada), mientras que las parametrizaciones y la experiencia de usuario se desarrollan bajo metodologías ágiles (Scrum/DevOps).

Entrega de Valor Continua

En lugar de esperar un año para el despliegue final, se aplican los principios de PMBOK para segmentar el proyecto en Productos Mínimos Viables (MVP). Esto garantiza que el negocio valide el software de manera constante, minimizando fallas críticas en producción.

Gestión de Interesados (Stakeholders)

Los proyectos de TI fallan principalmente por falta de adopción de los usuarios. El dominio de interesados del PMBOK define estrategias de comunicación efectiva entre los desarrolladores técnicos y los líderes de negocio para asegurar una transición fluida y mitigar la resistencia al cambio.

Datos de Valor y Recomendaciones

- **Certificación Profesional:** La certificación PMP (Project Management Professional) del PMI está alineada directamente con estos principios. Los profesionales certificados en TI suelen reportar salarios significativamente más altos debido a su capacidad demostrada para liderar equipos técnicos.
- **El Secreto es la Adaptación (Tailoring):** No intente implementar el PMBOK como si fuera un manual de instrucciones paso a paso. El éxito radica en el 'Tailoring' (adaptación): tome sólo los procesos, artefactos y métricas que generen valor real para su equipo de desarrollo.

PRINCE2 (PRojects IN Controlled Environments)

Desarrollado y promovido en el Reino Unido y ampliamente usado en Europa y organismos internacionales, PRINCE2 es un método estructurado para la gestión de proyectos centrado en el proceso. Se basa en 7 Principios, 7 Temáticas y 7 Procesos. Define claramente los roles y responsabilidades de gobernanza (Comité del Proyecto) y se asegura de que el proyecto mantenga una justificación continua del negocio (Business Case).

GUÍA PRÁCTICA DE PRINCE2 (Projects IN Controlled Environments)

Contexto, Importancia Actual y Aplicación en Proyectos de TI

Contexto General de PRINCE2

PRINCE2 (PRojects IN Controlled Environments) es un método estructurado de gestión de proyectos desarrollado originalmente por el gobierno del Reino Unido y actualmente administrado globalmente por AXELOS (un joint venture entre Capita y la Cabinet Office). A diferencia de marcos basados en guías de conocimiento (como el PMBOK del PMI), PRINCE2 es una metodología explícita paso a paso basada en procesos y de libre uso.

El método nació en 1989 bajo el nombre de PRINCE, enfocado inicialmente en proyectos de sistemas de información del gobierno británico. En 1996, tras una profunda revisión y simplificación orientada a volverlo aplicable a cualquier tipo de proyecto y sector, se lanzó de manera oficial como PRINCE2. Su última gran evolución se consolida en la versión PRINCE2 7 (lanzada a finales de 2023), la cual refuerza la agilidad, la sostenibilidad y la gestión de personas en entornos modernos.

¿Para qué sirve PRINCE2?

PRINCE2 sirve para proporcionar una estructura clara, repetible y escalable que garantice el control total de un proyecto desde su concepción hasta su cierre. Su objetivo fundamental es asegurar que el proyecto mantenga una justificación comercial válida en todo momento.

El método divide el proyecto en fases manejables, lo que permite un control eficiente de los recursos y una supervisión basada en excepciones. Esto significa que la alta dirección delega la gestión diaria al Director de Proyecto y solo interviene si se superan los límites de tolerancia preestablecidos (tiempo, coste, calidad, alcance, beneficios y riesgos).

La Estructura de PRINCE2

El marco de trabajo de PRINCE2 se fundamenta en un ecosistema integrado por cuatro elementos centrales:

Los 7 Principios (El 'Por Qué')

Son las leyes obligatorias que dictan si un proyecto está siendo gestionado bajo PRINCE2 o no:

- **1.** Justificación comercial continua: El proyecto debe tener un beneficio claro y un retorno de inversión real. Si deja de ser viable, se cancela inmediatamente.
- **2.** Aprender de la experiencia: Los equipos deben registrar, buscar y aplicar lecciones aprendidas de proyectos anteriores.
- **3.** Roles y responsabilidades definidos: Estructura organizativa clara que responde a tres áreas principales: negocio (patrocinador), usuario (quien lo usa) y proveedor (quien lo construye).
- **4.** Gestión por fases: Planificación, supervisión y control del proyecto bloque a bloque mediante hitos de decisión bien delimitados.

- **5.** Gestión por excepción: El comité de dirección establece tolerancias. Si el Director de Proyecto prevé superarlas, eleva el problema ('excepción') para su resolución.
- **6.** Enfoque en los productos: Los proyectos se centran en la definición, entrega y calidad de los entregables tangibles, no solo en la ejecución de tareas.
- **7.** Adaptación al entorno: El método debe dimensionarse según el tamaño, complejidad, importancia y nivel de riesgo del proyecto.

Las 7 Prácticas (El 'Qué')

Anteriormente llamadas 'Temáticas', representan los aspectos de la gestión del proyecto que requieren atención continua:

- **1.** Caso de Negocio (Business Case): El motor del proyecto; justifica el inicio y la continuidad técnica y económica.
- **2.** Organismo / Organización: Define la estructura del equipo (Comité de Proyecto, Director de Proyecto, Equipos de Trabajo).
- **3.** Calidad: Asegura que los productos cumplan con los requerimientos técnicos y criterios de aceptación del usuario.
- **4.** Planes: Describe las estrategias y cronogramas para alcanzar los objetivos en diferentes niveles (Proyecto, Fase, Equipo).
- **5.** Riesgo: Identifica, evalúa y controla la incertidumbre para proteger la viabilidad del proyecto.
- **6.** Cambio: Evalúa y gestiona de forma controlada cualquier modificación a la línea base o a los productos establecidos.
- **7.** Progreso: Mide los logros del proyecto frente a los planes para controlar la viabilidad de la continuidad.

Los 7 Procesos (El 'Cómo')

Representan el ciclo de vida del proyecto desde el inicio hasta el cierre definitivo:

- **1.** Puesta en marcha de un proyecto (Starting up a Project).
- **2.** Dirección de un proyecto (Directing a Project).
- **3.** Inicio de un proyecto (Initiating a Project).
- **4.** Control de una fase (Controlling a Stage).
- **5.** Gestión de la entrega de productos (Managing Product Delivery).
- **6.** Gestión de los límites de fase (Managing a Stage Boundary).
- **7.** Cierre de un proyecto (Closing a Project).

Importancia Actual de PRINCE2

En el entorno de negocios moderno, la relevancia de PRINCE2 radica en su capacidad para prevenir el desperdicio de capital. Muchas organizaciones fallan al continuar ejecutando proyectos que han perdido su valor estratégico. Al exigir una justificación comercial dinámica, PRINCE2 obliga a los comités directivos a pivotar o detener iniciativas inviables.

Asimismo, su enfoque en la definición explícita de roles reduce los conflictos de autoridad y la falta de rendición de cuentas, un problema recurrente en las transformaciones corporativas digitales.

Aplicación de PRINCE2 en Proyectos de TI

PRINCE2 nació en la tecnología de la información y sigue siendo uno de los marcos de gobernanza informática más potentes. A continuación se detalla cómo se aplica en la práctica:

Gobernanza de TI y Conexión Directa con Scrum

En el desarrollo de software moderno, PRINCE2 opera como la capa de 'Gobernanza' superior, mientras que metodologías como Scrum o Kanban se ejecutan en las células de desarrollo (capa técnica). Esto se formaliza a través del proceso 'Gestión de la Entrega de Productos':

- • El Director de Proyecto entrega un 'Paquete de Trabajo' (un bloque de funcionalidades requeridas).
- • El Product Owner / Scrum Master traduce este paquete en Historias de Usuario dentro del Product Backlog.
- • Las células de desarrollo ejecutan Sprints para construir el software bajo su propio ritmo y reglas ágiles.
- • El Scrum Team entrega un incremento potencialmente desplegable que cumple con los criterios de calidad de PRINCE2.

Gestión de Despliegues de Infraestructura y Migraciones en la Nube

Para proyectos críticos y de alto riesgo como la migración de un Datacenter a la nube (AWS/Azure), el principio de 'Gestión por Fases' es vital. PRINCE2 permite estructurar el proyecto en etapas lógicas:

- • Fase 1: Auditoría y diseño de arquitectura (Predictivo).
- • Fase 2: Migración piloto de aplicaciones no críticas (Híbrido/Ágil).
- • Fase 3: Migración masiva de bases de datos de producción (Predictivo de alta seguridad).

Cada fase requiere la aprobación explícita del Comité de Proyecto para liberar el presupuesto de la siguiente etapa.

Control estricto de Tolerancias contra el 'Scope Creep' Tecnológico

En proyectos de TI es común la corrupción del alcance (agregar funciones sin control). El principio de 'Gestión por Excepción' funciona de la siguiente manera: si el equipo técnico identifica que la integración de una API de terceros retrasará el despliegue más allá de la tolerancia de tiempo permitida (por ejemplo, 5 días), el Director de Proyecto no puede decidirlo solo; debe elevar un Reporte de Excepción al Comité para reevaluar la continuidad comercial de esa integración.

Datos Adicionales y Comparativa de Mercado

Criterio	PRINCE2	PMBOK (PMI)
Naturaleza	Metodología estructurada paso a paso (Procesos explícitos). El 'Cómo' gestionar el proyecto.	Guía de fundamentos de conocimiento (Buenas prácticas). El 'Qué' necesita saber un gestor.
Gobernanza	Define con precisión la toma de decisiones directiva (Comité de Proyecto).	Se enfoca fuertemente en las habilidades y dominios del Director de Proyecto.
Adaptabilidad	Altamente integrable con marcos ágiles mediante PRINCE2 Agile®.	Altamente flexible en su 7ma edición basada en Principios y Sistemas de Entrega de Valor.

Esquema de Certificaciones Oficiales

- • PRINCE2 Foundation: Valida el conocimiento de los términos, terminología y estructura básica del método.
- • PRINCE2 Practitioner: Certifica que el profesional posee la capacidad técnica para adaptar y aplicar el método a un escenario de proyecto real.
- • PRINCE2 Agile®: Una variante específica diseñada para organizaciones que operan completamente bajo metodologías ágiles en su capa técnica, proporcionando la gobernanza exacta sin burocracia.

Cuadro Comparativo: PMBOK vs. PRINCE2

Dimensión	PMBOK (PMI)	PRINCE2
Naturaleza	Guía de conocimientos y buenas prácticas. Describe "qué hacer" y las competencias del PM.	Metodología paso a paso. Describe "cómo hacer", estructuras de control y gobernanza.
Orientación	Centrado principalmente en las habilidades y conocimientos del Director del Proyecto.	Centrado en el proceso de toma de decisiones del Comité Directivo del Proyecto.

Dimensión	PMBOK (PMI)	PRINCE2
Justificación	El Acta de Constitución (Project Charter) inicia el proyecto.	Justificación continua del negocio mediante el Caso de Negocio (Business Case).
Adaptabilidad	Se adapta a las mejores prácticas de la organización.	Diseñado intrínsecamente para ser adaptado (Tailoring) a cada proyecto.

Estudios de Viabilidad Técnica, Operativa y Financiera

Antes de autorizar el inicio de un proyecto de TI, es indispensable realizar un análisis integral de viabilidad para garantizar que el proyecto sea realizable, sostenible y rentable. Dicho estudio depende a su vez de los recursos necesarios y disponibles para el proyecto.

Recursos para un proyecto

Todo proyecto en cualquier ámbito debe establecer y definir desde un principio qué recursos necesita para su creación e implementación. De esta forma se identifican los requisitos básicos que se añaden a la planeación, permiten iniciar su desarrollo y prever el cumplimiento de tiempos para su finalización. Esto a su vez, determina qué tan factible es llevar a cabo dicho proyecto. Veamos los recursos que intervienen en todo proyecto.

Humanos

Personas que participan en el proyecto, ya sea de forma directa o indirecta: ejecutores, clientes y los contactos de éste, asesores, directores, etc.

Técnicos / Tecnológicos

Corresponde a las herramientas tecnológicas (hardware, software, maquinaria) y un conjunto de técnicas que permiten el manejo adecuado de dicha tecnología, y que exige una experticia por parte de las personas encargadas del manejo de estos recursos. Mientras la tecnología se refiere a los componentes necesarios para operar un negocio, la técnica habla de las habilidades del personal para operar con dicha tecnología.

Económicos / Financieros

Corresponde al dinero (capital) con que se cuenta para llevar a cabo el desarrollo del proyecto. Incluye el costo de los recursos, los viáticos, honorarios, asesorías, etc.

Operativos

Se refiere a material de trabajo diario, tales como elementos de oficina (papelería, lápices, escritorios, tinta de impresión, etc.), libros y manuales. También puede incluir otros elementos de acuerdo a las necesidades, como productos especiales para limpiar ciertos componentes tecnológicos, entre otros.

Estudio de factibilidad / viabilidad de un proyecto

El estudio de factibilidad permite decidir qué tan viable (posible de realizar) es o no llevar a cabo un proyecto basados en los recursos con los que se cuenta; la factibilidad se analiza entonces desde lo humano, económico y tecnológico, respondiendo a preguntas que indican qué tan fluido o atascado será el camino para lograr los objetivos propuestos.

El estudio de factibilidad incluye aspectos tales como:

- **Evaluación de la viabilidad**: implica determinar si el proyecto puede ser realizado, considerando factores como la disponibilidad de recursos humanos, infraestructura, económicos, materiales, equipos, etc.
- **Análisis de la rentabilidad**: evalúa si los beneficios que se esperan obtener del proyecto son mayores que los costos de su implementación y operación.
- **Consideración de riesgos**: identifica y evalúa los posibles riesgos y obstáculos que podrían afectar el éxito del proyecto.
- **Beneficios esperados**: consiste en evaluar si el proyecto aportará valor a la organización, al mercado o a la sociedad en general.

Importancia de la factibilidad

Sin un estudio de factibilidad, un proyecto estaría a la deriva, sin planeación, lleno de improvisación y condenado al fracaso, ya que ésta precisamente:

- **Reduce el riesgo de fracaso**: al evaluar la factibilidad, se identifica y reduce el riesgo de que el proyecto no sea viable.
- **Mejora la toma de decisiones**: proporciona información para tomar decisiones informadas sobre si se debe o no llevar a cabo el proyecto.
- **Ahorra tiempo y dinero**: al evaluar la factibilidad, se evita invertir tiempo y dinero en proyectos que no son viables.
- **Mejora la planificación**: ayuda a planificar mejor el proyecto y a establecer metas realistas.

A nivel de proyectos de SI, es común analizar la factibilidad desde los aspectos, humano, técnico, tecnológico, económico y operacional. Veamos cada una de estas.

Factibilidad humana

¿Las personas conforman grupos cohesionados?

¿Los grupos conformados generan discusiones que no llevan a ningún lado?

¿El equipo es responsable y comprometido?

Factibilidad técnica/tecnológica

¿Los implementos tecnológicos permiten solucionar los requerimientos propuestos?

¿Las personas cuentan con los conocimientos suficientes para operar los recursos tecnológicos?

¿La tecnología a usar es obsoleta o permite actualizarse y se adapta a los requisitos?

Factibilidad económica

¿Se cuenta con el dinero suficiente para emprender y construir el proyecto?

¿Puede la organización asumir la inversión y los costos de un proyecto?

Factibilidad operacional

¿Los empleados son reacios al nuevo sistema?

¿El nuevo sistema mejora la productividad de la organización?

¿Los colaboradores se adaptarán fácilmente al cambio o éste será traumático?

Nota

Los recursos y la factibilidad es de lo primero en analizar al iniciar un proyecto; por tanto esta etapa se emprende antes de entrar en el análisis del problema a resolver, esto es, antes de la construcción de diagramas y el estudio de los procesos que éstos involucran.

Tipos de Viabilidad en gestión de proyectos

Los tipos de viabilidad estudiados arriba aplican en la gerencia de proyectos y no solo en proyectos de TI, y le da especial relevancia a los siguiente tipos:

- **Viabilidad Técnica:** Evalúa si la organización cuenta con la infraestructura, tecnologías, arquitectura de software y capacidades técnicas necesarias (o si se pueden adquirir) para construir el sistema.
- **Viabilidad Operativa:** Evalúa si la solución se integrará efectivamente en los procesos del negocio, la aceptación por parte de los usuarios finales (gestión del cambio) y el cumplimiento de las normativas.
- **Viabilidad Financiera:** Evalúa la relación costo-beneficio del proyecto para determinar si los retornos económicos justifican la inversión de capital.

Framework XP (Extreme Programming) en Proyectos de TI

Guía Detallada de Conceptos, Prácticas e Implementación

Introducción a Extreme Programming (XP)

Extreme Programming (XP) es una de las metodologías ágiles más destacadas en el desarrollo de software. Fue formulada por Kent Beck a finales de la década de 1990 con el objetivo de mejorar la calidad del producto final y la capacidad de respuesta ante los requerimientos cambiantes de los clientes. XP se centra en potenciar las relaciones interpersonales como clave del éxito, promoviendo el trabajo en equipo, el aprendizaje continuo y un ambiente laboral propicio.

Los 5 Valores Fundamentales de XP

El éxito de la aplicación de XP radica en la adopción estricta de sus cinco valores esenciales:

- **Comunicación:** El equipo mantiene una comunicación constante y abierta, tanto de forma interna como con el cliente, minimizando malentendidos.
- **Simplicidad:** Se busca la solución más limpia y directa que funcione, evitando la sobreingeniería o el desarrollo de funcionalidades futuras no requeridas en el presente.
- **Retroalimentación (Feedback):** Se entrega software funcional de manera temprana y frecuente para obtener una retroalimentación directa del cliente y corregir el rumbo rápidamente.
- **Coraje (Valor):** Los desarrolladores deben ser valientes para aceptar los cambios en los requisitos, desechar código obsoleto o refactorizar arquitecturas complejas cuando sea necesario.
- **Respeto:** Se fomenta un respeto mutuo entre programadores, administradores de proyectos y clientes. Todos los aportes son valiosos para el fin común.

Las 12 Prácticas Clave de XP

XP operacionaliza sus valores a través de doce prácticas de ingeniería y gestión de proyectos:

- **Juego de la Planificación (Planning Game):** El cliente define las funcionalidades que desea mediante 'historias de usuario', y junto al equipo técnico, estiman el esfuerzo y planifican las entregas.

- **Entregas Pequeñas (Small Releases):** El software se entrega a los usuarios finales en periodos de tiempo muy cortos (semanas), permitiendo validar el valor real del producto.
- **Metáfora del Sistema:** Se utiliza una terminología común y metáforas sencillas para describir la arquitectura del sistema, asegurando que todos entiendan el diseño global.
- **Diseño Simple:** El diseño del código debe mantenerse lo más simple posible en todo momento, respondiendo solo a las necesidades actuales.
- **Desarrollo Guiado por Pruebas (TDD):** Consiste en escribir las pruebas automatizadas unitarias antes de codificar la funcionalidad real, garantizando la cobertura de pruebas y un diseño modular.
- **Refactorización:** Proceso continuo de reestructuración del código existente para mejorar su legibilidad y mantenerlo limpio, sin alterar su comportamiento externo.
- **Programación en Parejas (Pair Programming):** Dos programadores trabajan juntos en la misma estación de trabajo. Uno escribe el código ('conductor') mientras el otro revisa y piensa estratégicamente ('navegante').
- **Propiedad Colectiva del Código:** Cualquier miembro del equipo tiene el derecho y la responsabilidad de modificar cualquier parte del código para mejorarlo o corregir errores.
- **Integración Continua (CI):** El código desarrollado por las parejas se integra y se prueba en la rama principal varias veces al día, reduciendo drásticamente los conflictos de integración.
- **Ritmo Sostenible (40 horas semanales):** El ritmo de trabajo debe ser sostenible en el tiempo. Se evitan las horas extra sistemáticas para mitigar el agotamiento y mantener la calidad técnica.
- **Cliente en el Sitio (On-site Customer):** Un representante del cliente o usuario final trabaja directamente con el equipo de desarrollo a tiempo completo para resolver dudas y validar el producto.
- **Estándares de Código:** El equipo define y respeta un estándar de codificación común para asegurar que todo el código parezca escrito por una sola persona.

Cómo Aplicar XP en Proyectos de TI

La implementación de Extreme Programming en un proyecto de tecnología requiere una transformación cultural y técnica estructurada en las siguientes fases operativas:

Fase 1: Preparación y Descubrimiento

- Definir un espacio físico o virtual compartido que facilite la comunicación visual directa (tableros Kanban digitales, salas de comunicación abierta).
- Capacitar al equipo en prácticas técnicas esenciales, especialmente TDD, diseño simple y herramientas de integración continua.
- Identificar e integrar al 'Cliente en el Sitio' para la redacción de las primeras historias de usuario.

Fase 2: Planificación del Release (Lanzamiento)

- Desglosar el alcance inicial en historias de usuario cortas con criterios de aceptación claros.
- El equipo técnico estima el esfuerzo (puntos de historia o días ideales) sin interferencia del cliente.
- El cliente prioriza las historias basándose en el valor de negocio, definiendo el alcance del primer release.

Fase 3: Ciclos de Iteración (Semanales o Quincenales)

- Reunión diaria de sincronización (Daily Standup) de máximo 15 minutos para coordinar el trabajo del día y remover bloqueos.
- Desarrollo diario aplicando estrictamente la programación en parejas (Pair Programming) y rotando los roles de forma regular.
- Ejecución del flujo TDD: escribir la prueba (Fallo), escribir el código mínimo para pasarla (Éxito) y refactorizar.

Fase 4: Integración y Despliegue Continuo

- Automatizar el pipeline de CI/CD para que cada commit corra la suite completa de pruebas unitarias y de integración.
- Si una prueba falla en la integración, todo el equipo se enfoca en resolver el problema de inmediato.

Fase 5: Cierre de Iteración y Retroalimentación

- Demostración del software funcional al cliente al finalizar la iteración para recibir feedback inmediato.
- Reunión retrospectiva corta para analizar qué prácticas funcionaron y cuáles deben ajustarse en el siguiente ciclo.

- **Portal de Comunidad:** Agile Alliance. 'Extreme Programming (XP)'. Disponible en su portal oficial (agilealliance.org). Repositorio de recursos y artículos detallados sobre la evolución de XP en la industria actual.
- **Recursos de Gestión:** Mountain Goat Software / Mike Cohn. Artículos y guías sobre planeación y estimación ágil adaptables al juego de la planificación de XP.

¿Por qué evitar la división tradicional de Caja Blanca / Caja Negra en TDD?

1. Confusión conceptual: TDD altera la forma clásica de pensar en pruebas. En TDD, la prueba diseña el código, no solo lo verifica.
2. Caja Negra vs. TDD: TDD nace desde la perspectiva de la especificación (¿Qué tiene que hacer este componente?). Escribes la prueba mirando la interfaz pública (caja negra).
3. Caja Blanca vs. TDD: Al escribir la prueba *antes*, no existe código que inspeccionar (no hay caja blanca). Sin embargo, como el desarrollador es quien escribe el código inmediatamente después, conoce perfectamente las tripas del sistema. TDD borra esa frontera tradicional.
4. Foco en el flujo XP: En XP, el TDD se vive a través del ciclo Red-Green-Refactor.

Red-Green-Refactor

Para entender a fondo el ciclo **Red-Green-Refactor**, es útil ver de dónde nace, por qué rompe con la programación tradicional y cuáles son las reglas estrictas que lo hacen funcionar.

🧠 La Filosofía: Cambiar el "Modo de Pensar"

En el desarrollo de software tradicional, primero escribes el código y luego lo pruebas. El TDD (creado por Kent Beck en los años 90) le da la vuelta por completo a este proceso.

El ciclo divide tu cerebro en tres roles distintos para que no intentes resolver todo a la vez:

1. **El Diseñador (Red):** Piensa en qué debe hacer el programa desde el punto de vista del usuario o de quien consumirá ese código. Define la interfaz ideal.
2. **El Hacker (Green):** Solo quiere resolver el problema rápido. No le importa si el código es feo, usa un parche o es ineficiente, siempre y cuando funcione.
3. **El Artesano (Refactor):** Aplica buenas prácticas, elimina código duplicado, mejora los nombres de las variables y optimiza el rendimiento.

Las 3 Reglas del TDD (Por Uncle Bob)

Robert C. Martin (Uncle Bob), uno de los autores del Manifiesto Ágil, resumió la disciplina del ciclo rápido en tres leyes estrictas:

- **Primera Ley:** No tienes permitido escribir ningún código de producción a menos que sea para hacer pasar una prueba unitaria que está fallando.
- **Segunda Ley:** No tienes permitido escribir más de una prueba unitaria de lo necesario para fallar; y no compilar es fallar.
- **Tercera Ley:** No tienes permitido escribir más código de producción de lo necesario para hacer pasar la prueba que falla.

Esto significa que el ciclo no dura horas, ¡dura minutos o incluso segundos por cada iteración!

Los Beneficios Reales (Los Trade-offs)

Adoptar este ciclo rápido tiene ventajas masivas, pero también exige un precio:

Ventajas 🚀	Desventajas / Retos ⚠️
Cero miedo al cambio: Puedes cambiar cualquier parte del sistema y saber en segundos si rompiste algo.	**Curva de aprendizaje alta:** Escribir pruebas antes del código se siente antinatural al principio y requiere práctica.
Diseño modular: Como el código debe ser fácil de probar, por defecto se vuelve modular, desacoplado y limpio.	**Mayor tiempo inicial:** Escribir pruebas toma tiempo al inicio del proyecto (aunque te ahorra semanas de debugging después).
Documentación viva: Las pruebas de la fase *Red* sirven como la mejor documentación posible de cómo debe funcionar el código.	**Mantenimiento mutuo:** Si los requerimientos del negocio cambian, debes actualizar tanto las pruebas como el código.

Un Ejemplo Mental Rápido

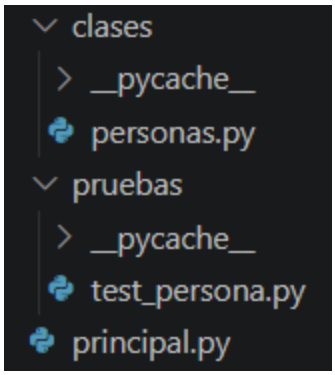
Imaginemos que se va a crear una función para validar si una contraseña es segura (mínimo 8 caracteres):

1. **Red:** Se escribe un test que pasa una contraseña corta (`"123"`) esperando un

resultado `falso`. Se ejecuta el test y falla (da error porque la función ni siquiera existe).

2. **Green:** Crear la función y se pone literalmente `return false;`. Ejecutar el test y ¡pasa! (Vuelve a verde). Has hecho lo mínimo estrictamente necesario.
3. **Refactor:** Agregar un segundo test con una contraseña larga (`"password123"`), vuelve a fallar (*Red*), cambias la función para que evalúe la longitud (`if largo > 8`), el test pasa (*Green*). Si el código quedó desordenado, se limpia (*Refactor*).

Ejemplo



Paso 1: El estado RED (Escribir la prueba antes que la clase)

Creamos `test_persona.py`. Diseñamos la interfaz pública de nuestra clase desde la perspectiva de la "caja negra": decidimos cómo se llamará la clase, cómo se añaden productos y qué método calcula el total. El código de producción aún no existe.

principal.py

```
from clases.pruebas_tdd import Persona

per = Persona()

nom = input("Nombre: ")
ed = int(input("Edad: "))

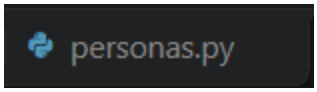
per.set_nombre(nom)
```

```
per.set_edad(ed)

print("Datos persona")
print(f"{per.get_nombre()}, usted es {per.categoria_edad()} y " \
      f"tiene {per.get_edad()} años")
```

Paso 2: El estado GREEN (Hacer que pase con lo mínimo)

Creamos el archivo `personas.py` e implementamos la estructura de la clase con el código mínimo absoluto (incluso con respuestas fijas) para obligar a los tests a ponerse en verde.




```
class Persona():

    def __init__(self):
        self.nombre = ""
        self.edad = 0
        self.salario = 0

    def get_nombre(self) -> str:
        return self.nombre

    def get_edad(self) -> int:
        return self.edad

    def get_salario(self) -> int:
        return self.salario

    def set_nombre(self, nombre) -> None:
        self.nombre = nombre

    def set_salario(self, salario) -> None:
```

```
        self.salario = salario

    def set_edad(self, edad) -> None:
        self.edad = edad

    def categoria_edad(self) -> str:
        if self.edad < 14:
            return "Niño"
        elif self.edad <= 28:
            return "Joven"
        else:
            return "Adulto"

    def verificar_mayoria_edad(self, edad) -> bool:
    #     if edad >= 18:
            return True
        # return False

    def bonificacion_salario(self, bonificacion) -> float:
        return 10
```

Resultado al ejecutar: FAIL / ERROR (`TypeError: 'NoneType' object is not callable` o `ImportError`). Estamos en **RED**.

Paso 3: Agregar una prueba adicional (Volvemos a RED)

Ahora agregamos el requerimiento del negocio.

test_persona.py

```
import sys
import os
# Detecta la carpeta raíz del proyecto y la agrega al buscador de Python
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))
```

```python
import unittest

# Intentar importar algo que todavía no existe de la caja negra llamada
"Personas"
try:
    from clases.pruebas_tdd import Persona
except ImportError:
    verificar_mayoria_edad = None

class TestPersonaTDD(unittest.TestCase):

    def test_mayoria_edad(self):
        persona = Persona()
        self.assertTrue(persona.verificar_mayoria_edad)

    def test_bonificacion_salario(self):
        persona = Persona()
        self.assertEqual(persona.bonificacion_salario(100), 10)

if __name__ == '__main__':
    unittest.main()
```

Resultado al ejecutar el test: GREEN (OK). La prueba pasa.

Ejecución

1
```
>py .\pruebas\test_persona.py
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
```

2
```
>py -m unittest test_persona.py
```

```
>py .\pruebas\test_persona.py -v
test_bonificacion_salario (__main__.TestPersonaTDD.test_bonificacion_salario) ... ok
test_mayoria_edad (__main__.TestPersonaTDD.test_mayoria_edad) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.001s

OK
```

Paso 4: Implementar la lógica real de POO para volver a GREEN

Modificamos `personas.py`. Ahora sí estructuramos los atributos internos (una lista y/o diccionario) y resolvemos los requerimientos.

Paso 5: REFACTOR (Limpiar el diseño orientado a objetos)

El código funciona. Refactorizamos encapsulando las reglas de negocio en métodos privados y aplicando buenas técnicas para mejorar el código.

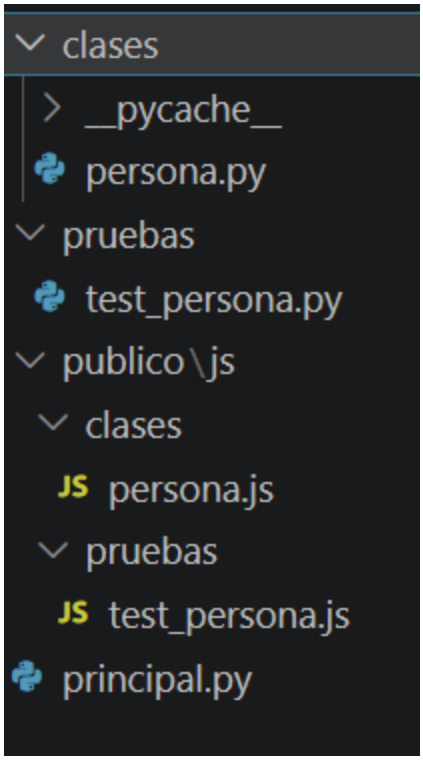
Nota

Resumiendo algunos conceptos:

- **Scrum** da el marco de trabajo (el "**qué**" y "**cuándo**"), **XP** da la ingeniería para mantener el código limpio y sostenible (el "**cómo**")
- La prueba es un **contrato de diseño.**
- Ciclo rápido: ***Red*** (falla), ***Green*** (pasa con lo mínimo), ***Refactor*** (limpia).

Pruebas con Javascript

Estructura del proyecto:



JS persona.js

```
export class Persona {

    constructor(nombre, edad, salario) {

        this.nombre = nombre;

        this.edad = edad;

        this.salario = salario;

    }

    verificar_mayoria_edad() {

        return this.edad >= 18;

    }
```

```
    bonificacionSalario(bonificacion) {

        if (this.salario > 0) {

            return bonificacion * 100 / self.salario;

        }

        return 0;

    }

}
```

JS test_persona.js

```
import test from 'node:test';

import assert from 'node:assert';

import { Persona } from '../clases/persona.js' // Importación directa

test('Una persona de 25 años debe ser mayor de edad', () => {

    // Preparar (Arrange)

    const persona = new Persona("Ana", 25, 2000000);

    // Actuar (Act)

    const resultado = persona.verificar_mayoria_edad();

    // Afirmar (Assert)

    assert.strictEqual(resultado, true);

});
```

```
test('Una persona de 10 años no es mayor de edad', () => {

    const persona = new Persona("Hugo", 10, 0);

    assert.strictEqual(persona.verificar_mayoria_edad(), false)

});
```

Ejecución de las pruebas

>node --test pruebas/test_persona.js

Ejercicio

Ejecute la siguiente secuencia para el historial de commits o vinculada a las herramientas de gestión de proyectos (como Jira, GitHub Issues o Trello): con esto se obtiene visibilidad total de cómo TDD rastrea el progreso real del desarrollo.

1. `git commit -m "feat: test para validador (RED)"`
2. `git commit -m "feat: implementación básica validador (GREEN)"`
3. `git commit -m "refactor: limpieza de código de validación (REFACTOR)"`

Del Tablero al Código: tomar la tarea más pequeña, simple e insignificante que tengan pendiente en el sprint actual del proyecto. Olvidarse de la base de datos o interfaces web; buscar **lógica de negocio pura** (una validación, un cálculo, un formateo).

Programación en Parejas (Pair Programming): Siendo XP, se debe trabajar en parejas. Uno escribe la prueba (el *Navigator* dicta) y el otro escribe el código para que pase (el *Driver* ejecuta).

Puesta en común: ¿Qué dolió más? ¿Por qué costó tanto no escribir el código primero? ¿Cómo se siente al ver la barra en verde antes de dar por terminada la tarea?

El Puente entre Scrum y XP: De Criterios de Aceptación a Tests

Historia de Usuario: seleccione una historia de usuario; ejemplo: *"Como usuario,*

quiero registrarme en la plataforma..."

Criterio de Aceptación (Scrum): por ejemplo: *"La contraseña debe tener al menos 8 caracteres"*.

El Enfoque XP (TDD): Ese criterio se traduce directamente en un método de prueba en su archivo: `def test_contrasena_corta_rechazada(self):`.

Programación en Parejas (Pair Programming) "Micro-estilo"

Trabajar en parejas bajo la dinámica **Ping-Pong TDD**:

- **Desarrollador A:** Escribe una prueba que falla (**RED**).
- **Desarrollador B:** Escribe el código mínimo para que pase (**GREEN**) y hace el **REFACTOR**.
- **Desarrollador B:** Ahora le toca a él escribir la siguiente prueba que falla (**RED**).
- **Desarrollador A:** Escribe el código para que pase (**GREEN**)... y el ciclo se repite.

Integración Continua (CI) Conceptual

En XP, el código se integra varias veces al día.

Integración Continua Manual de 2 reglas:

1. Nadie puede hacer `git merge` o subir código a la rama principal si el comando `py .\pruebas\test_persona.py` (o su equivalente) da un solo error.
2. Si un desarrollador rompe las pruebas en la rama principal, su única prioridad absoluta es arreglarlas inmediatamente.

Propiedad Colectiva del Código

Al tener una red de pruebas (como el `test_persona.py`), **cualquier miembro del equipo puede modificar cualquier parte del código con total confianza**. Si alguien cambia algo en la clase de otro compañero y rompe la lógica, el test saltará de inmediato. Esto elimina el cuello de botella de *"ese módulo solo lo toca Frumencio porque él lo programó"*.

Un tablero Scrum con el sprint final (con el que venían haciendo).

Una suite de pruebas básicas (mínimo 10 pruebas unitarias core de la lógica de negocio, no de la base de datos).

Evidencia en los commits de Git donde se vea que aplicaron programación en parejas y que las pruebas corrían antes de integrar el código.

Nota

Entender qué hace TDD no es "inventar pruebas", es automatizar los Criterios de Aceptación que ellos mismos definieron en su Product Backlog.

Integración Continua (CI) con GitHub Actions

Para la **Integración Continua (CI)** usaremos **GitHub Actions**, el cual ejecuta servidores virtuales (runners) capaces de correr código backend, bases de datos y ejecutar tus pruebas automatizadas de Python y JavaScript sin ningún problema ni costo para repositorios públicos.

Guía Paso a Paso para configurar la CI en GitHub

Estructuraremos la Integración Continua utilizando **GitHub Actions**. Se necesita crear un archivo de configuración en una ruta específica del repositorio para que GitHub ejecute los tests automáticamente cada vez que se haga un `push` o un `Pull Request`.

1. Estructura recomendada del repositorio

Organizar el proyecto de la siguiente manera (pueden coexistir ambos lenguajes):

text

```
None
mi-proyecto-xp/
├── .github/
│   └── workflows/
│       └── ci.yml          <-- ESTE ES EL ARCHIVO DE CONFIGURACIÓN DE CI
├── python_project/
│   ├── src/
│   ├── tests/              <-- Pruebas unitarias de Python (TDD)
│   └── requirements.txt    <-- Dependencias de Python (si aplica)
└── javascript_project/
    ├── src/
    ├── tests/              <-- Pruebas unitarias de JS (TDD)
    └── package.json        <-- Dependencias y scripts de JS
```

2. Crear el archivo de configuración de CI (`ci.yml`):

`.github/workflows/ci.yml`

Dentro de la carpeta `.github/workflows/`, crear un archivo de texto llamado `ci.yml`. Este archivo le dice a GitHub qué entorno virtual levantar y qué comandos ejecutar (similar al `Jenkinsfile` que se usa en Jenkins).

Copiar y pegar el siguiente contenido balanceado para ambos lenguajes:

Este archivo está optimizado para buscar los archivos de `unittest` y ejecutar el comando nativo de pruebas de Node.js:

yaml

```
None
name: Integración Continua - Pruebas TDD (XP)


on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]


jobs:
  # TRABAJO 1: Ejecutar Pruebas de Python (unittest)
  test-python:
    runs-on: ubuntu-latest
    steps:
    - name: Descargar código del repositorio
      uses: actions/checkout@v4
```

```
    - name: Configurar entorno de Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.10'

    - name: Ejecutar Tests de Python con unittest
      run: |
        cd python_project
        # Descubre y ejecuta automáticamente todos los archivos que empiecen por test_
        python -m unittest discover -s tests -p "test_*.py"

  # TRABAJO 2: Ejecutar Pruebas de JavaScript (Node.js test runner)
  test-javascript:
    runs-on: ubuntu-latest
    steps:
    - name: Descargar código del repositorio
      uses: actions/checkout@v4

    - name: Configurar entorno de Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '20' # Versión estable con soporte nativo de test runner
```

```
    - name: Instalar dependencias si existen
      run: |
        cd javascript_project
        if [ -f package.json ]; then npm install; fi

    - name: Ejecutar Tests de JavaScript
      run: |
        cd javascript_project
        # Ejecuta el script de pruebas definido en tu package.json
        npm test
```

Nota: si los comandos para correr pruebas cambian (por ejemplo, si no usan `pytest` o si en el `package.json` el script de test es distinto), solo deben modificar la última línea de cada trabajo (`run`).

---

3. Preparación de los proyectos locales (Para que no falle la CI)

Antes de hacer el primer `git push`, asegurarse de que la estructura interna cumpla con los estándares de cada ejecutor:

En Python (`python_project/`):

- Dentro de tu carpeta `tests/`, asegurarse de incluir un archivo vacío llamado `__init__.py`. Esto le indica a Python que la carpeta es un paquete y permite que `unittest discover` encuentre las pruebas.
- Los archivos de prueba deben empezar por la palabra `test_` (por ejemplo: `test_calculadora.py`).

En JavaScript/Node.js (`javascript_project/`):

Asegurarse de que el archivo `package.json` tenga mapeado el comando de pruebas

nativo en la sección de `scripts`.

Crear el archivo `package.json`

Abrir la terminal, posicionarse dentro de la carpeta `javascript_project/` y ejecutar:

```
>npm init -y
```

Esto creará un archivo `package.json` básico automáticamente.

Para los requerimientos del ejercicio, tendrá la siguiente configuración:

{} package.json U

```
{
  "name": "javascript-project",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "test": "node --test tests/"
  }
}
```

(Nota: El comando `node --test` ejecuta automáticamente los archivos dentro de la carpeta `tests/` que terminen en `.test.js` o `test.js` sin necesidad de librerías externas).

---

4. Lista de verificación para la prueba:

1. Crear el repositorio en GitHub (público para que Actions sea 100% gratis e ilimitado).
2. Clonarlo en el computador, añadir los archivos de código, pruebas de TDD y la carpeta `.github/workflows/ci.yml`.
3. Hacer el commit y empujar los cambios:

bash

```
None

git add .

git commit -m "feat: agregar estructura base y configuracion de CI"

git push origin main
```

4. Ir directo a la pestaña **Actions** en la web de GitHub para ver cómo los dos trabajos (`test-python` y `test-javascript`) se ejecutan en paralelo.

4. Flujo de operación (Dinámica XP)

Una vez que el archivo `ci.yml` esté en el repositorio, la magia de la CI ocurre de forma automática:

1. **Ciclo TDD local:** Escribir una prueba que falla en la máquina local, escribir el código mínimo para que pase, y hacer la refactorización.
2. **Push a GitHub:** Subir los cambios (`git push origin main`), GitHub detectará inmediatamente el archivo en `.github/workflows/`.
3. **Monitoreo en la interfaz:**
    - Ir a la pestaña **"Actions"** en la barra superior de su repositorio de GitHub.
    - Allí se verá el *Workflow* ejecutándose en tiempo real.
    - Si sale un check verde ✅, la integración fue exitosa (el código respeta el diseño de XP).
    - Si sale una equis roja ❌, significa que se rompió algo. Al hacer clic en el error, se verá el desglose de la consola exactamente igual que en

Jenkins, permitiendo corregir el fallo de inmediato.

Fuentes Consultadas y Lecturas Recomendadas

Para profundizar en el conocimiento y la adopción práctica de Extreme Programming, se sugiere consultar las siguientes fuentes de referencia en el ámbito de la ingeniería de software:

- **Libro Guía:** Beck, K. (2004). 'Extreme Programming Explained: Embrace Change' (2nd Edition). Addison-Wesley Professional. El libro fundacional donde el creador de la metodología explica la filosofía y las prácticas.
- **Referencia Técnica:** Fowler, M. (2018). 'Refactoring: Improving the Design of Existing Code' (2nd Edition). Addison-Wesley. Un texto indispensable para comprender y aplicar la práctica de la refactorización dentro del flujo XP.

Indicadores Financieros de Evaluación de Proyectos

Para determinar la rentabilidad financiera de un proyecto TI, se utilizan tres métricas fundamentales:

ROI (Retorno de la Inversión / Return on Investment)

Mide el porcentaje de rendimiento obtenido sobre el capital invertido.
Fórmula:

ROI = ((Beneficios Totales − Inversión Total) / Inversión Total) × 100

VAN (Valor Actual Neto) / NPV (Net Present Value)

Es la suma de los flujos de caja futuros generados por el proyecto, descontados a una tasa de descuento (tasa de oportunidad o **WACC**) menos la inversión inicial.
Fórmula:

$VAN = \sum [FC_t / (1 + r)^t] - I_0$

Ejemplo en Python

```
# Cálculo VAN tomando datos de un vector
print("Cálculo VAN")
FC = [10, 50, 15, 25, 30]
VAN = 0
I0 = 10000
r = 0.1
for t in range(0, len(FC), 1):
    VAN += FC[t] / (1 + r) ** t
VAN = VAN - I0
print(f"VAN = {VAN}")
```

1. ¿En qué unidad se toma el tiempo t?

- **Por lo general**, en proyectos de inversión a mediano o largo plazo, t **se mide en años** (t = 1, 2, 3, …, n años).
- Sin embargo, el tiempo **puede medirse en meses, trimestres, semestres o días**, según el nivel de detalle de la evaluación o del tipo de negocio.
- **Regla de oro:** La unidad de tiempo de t **debe coincidir exactamente** con la unidad de tiempo de la tasa r y de los flujos de caja FC_t.

2. FC_t **es el flujo de caja**

- **Si el análisis es mensual:** FC_t es el flujo de caja del **mes** t (por ejemplo, $t = 1$ para enero, $t = 2$ para febrero, hasta $t = 12$ para diciembre).
- **Si el análisis es anual:** FC_t es el flujo de caja total generado durante todo el **año** t.

3. ¿Qué es r **y qué valor debe tomar?**

r **es la tasa de descuento** (también llamada tasa de oportunidad de capital, *WACC*, o tasa requerida de retorno).

Para determinar qué valor poner en r, debes tener en cuenta dos factores principales:

Equivalencia periódica (Consistencia temporal)

La tasa r debe expresarse en la misma unidad de tiempo que t:

- Si t está en **años**, r debe ser la **tasa efectiva anual**.
- Si t está en **meses**, r debe ser la **tasa efectiva mensual**.

Ejemplo de conversión:

Si tienes una Tasa Efectiva Anual (TEA) del 12% $(0,12)$ y tu análisis de flujos de caja FC_t es **mensual**, debes calcular la tasa efectiva mensual (TEM) equivalente mediante la fórmula de tasas equivalentes:

$$r_{mensual} = (1 + TEA)^{1/12} - 1$$

(1 + 0,12)^(1/12) - 1 = 0,009488 ó 0,9488% mensual

¿De dónde sale el valor de la tasa r?

El valor de r representa el costo del dinero o el rendimiento mínimo exigido a la inversión. Suele determinarlo:

1. **La tasa de oportunidad del inversionista:** Lo que ganaría invirtiendo ese dinero en la mejor alternativa de riesgo similar (por ejemplo, una renta fija, CDT, acciones, etc.).
2. **El costo de capital (WACC):** Si el proyecto se financia con deuda bancaria y capital propio, es el promedio ponderado del costo de ambas fuentes de financiamiento.
3. **El riesgo del proyecto:** A mayor riesgo o incertidumbre del proyecto, mayor será la tasa de descuento r exigida.

Ejemplo

Suponga que inviertes $10.000 ($I_0$) y esperas recibir flujos netos durante 3 meses:

- **Mes 1 ($t = 1$):** $4.000
- **Mes 2 ($t = 2$):** $4.000
- **Mes 3 ($t = 3$):** $4.000
- **Tasa mensual requerida (r):** 1% mensual $(0,01)$

El cálculo del VAN sería:

$$VAN = \frac{4000}{(1+0,01)^1} + \frac{4000}{(1+0,01)^2} + \frac{4000}{(1+0,01)^3} - 10000$$

TIR (Tasa Interna de Retorno) / IRR (Internal Rate of Return)

Es la tasa de descuento (r) que hace que el Valor Actual Neto (VAN) del proyecto sea igual a cero.
Ecuación para hallar la TIR:
$0 = \sum [FC_t / (1 + TIR)^t] - I_0$

La TIR es la tasa de rendimiento interna porcentual del proyecto.

Criterio de decisión: Se compara la TIR contra la tasa de oportunidad r.

- Si $TIR > r$, el proyecto es rentable (VAN > 0).
- Si $TIR < r$, el proyecto no cumple con el retorno esperado (VAN < 0).

A diferencia del VAN, en la TIR la tasa no la propones tú ni depende de una variable externa previamente dada.

La TIR (r_{TIR}) es el resultado desconocido que estás buscando resolver. Depende únicamente e independientemente de los propios datos del proyecto:

1. La Inversión Inicial (I_0).
2. La serie de Flujos de Caja (FC_t) a lo largo del tiempo.

1. ¿Qué representa la TIR exactamente?

La TIR es la tasa de rentabilidad o retorno porcentual intrínseca (propia) del proyecto.

Matemáticamente, la TIR es la tasa exacta r que hace que el costo inicial de la inversión sea

exactamente igual al valor presente de todos los ingresos futuros esperados:

$$I_0 = \sum_{t=1}^{n} \frac{FC_t}{(1+TIR)^t}$$

O expresado de otra forma, es la tasa r donde:

$$VAN = 0$$

2. ¿De qué depende la TIR?

Como la TIR es una propiedad interna del proyecto:

- Depende directamente de:
  - El monto de la inversión inicial (I_0): Si necesitas más dinero al inicio para ganar los mismos flujos futuros, la TIR baja.
  - La magnitud de los flujos de caja (FC_t): Si logras generar más dinero en cada periodo, la TIR sube.
  - El tiempo o velocidad con la que retornan los flujos (t): Si recibes los flujos más rápido (en los primeros periodos $t = 1, 2$), la TIR sube, porque el dinero temprano vale más que el dinero tardío.
- NO depende de:
  - La tasa de descuento del mercado o del inversionista (r).
  - La inflación.
  - Los costos de oportunidad externos.

3. Comparación conceptual entre VAN y TIR

Concepto	**En la fórmula del VAN**	**En la fórmula de la TIR**
Inversión Inicial (I_0)	Es un dato de entrada (conocido).	Es un dato de entrada (conocido).

Flujos de Caja (FC_t)	Son datos de entrada (conocidos).	Son datos de entrada (conocidos).
Tasa de descuento (r)	Es un dato de entrada (fijado de antemano por el mercado/inversionista).	¡Es la incógnita a calcular! (La TIR es la r desconocida).
Resultado obtenido	Un valor en dinero ($/€) (ganancia neta sobre lo exigido).	Un valor en porcentaje (%) (rentabilidad del proyecto).

4. ¿Cómo se usa la TIR para tomar decisiones?

Se puede usar un algoritmo o programa que encuentre el valor de la TIR (por ejemplo, TIR = 15% mensual), se compara contra la tasa de oportunidad (r) que el inversionista o la empresa exigía (por ejemplo, r = 10% mensual):

- Si $TIR > r$ (15% > 10%): El proyecto rinde más de lo mínimo exigido. Se acepta.
- Si $TIR = r$ (15% = 15%): El proyecto rinde exactamente lo exigido. Punto de equilibrio.
- Si $TIR < r$ (8% < 10%): El proyecto rinde menos de lo exigido. Se rechaza.

Planificación Macro, Gestión de Stakeholders y Líneas Base

Planificación Macro

Consiste en definir los límites generales del proyecto, los hitos estratégicos principales y los entregables de alto nivel antes de entrar en la descomposición detallada de tareas. Sirve para alinear la visión del proyecto con el sponsor y la alta dirección.

Gestión de Interesados (Stakeholders)

Identificación, análisis e involucramiento estratégico de todas las personas, grupos u organizaciones que pueden afectar o verse afectados por las decisiones del proyecto.
Matriz Poder e Interés: Herramienta clave para priorizar la comunicación y gestión de stakeholders:

Poder / Interés	Bajo Interés	Alto Interés
Alto Poder	**Mantener Satisfechos:** Cumplir sus expectativas sin saturar de detalles.	**Gestionar Atentamente (Clave):** Involucrar activamente en decisiones clave.
Bajo Poder	**Monitorear:** Mínimo esfuerzo con seguimiento periódico.	**Mantener Informados:** Comunicación constante sobre avances y cambios.

Triángulo de Restricciones y Líneas Base (Alcance, Tiempo, Costo)

Una **Línea Base (Baseline)** es la versión aprobada de un elemento de planificación que sirve como punto de referencia formal para medir y controlar el desempeño real del proyecto.

- **Línea Base del Alcance:** Compuesta por la Declaración del Alcance, la Estructura de Descomposición del Trabajo (EDT/WBS) y el Diccionario de la EDT.
- **Línea Base del Cronograma (Tiempo):** Cronograma aprobado con fechas de inicio y fin para cada paquete de trabajo e hitos.

- **Línea Base del Costo:** Presupuesto aprobado distribuido en el tiempo (Curva S).

Gestión Tradicional de Riesgos y Matriz de Riesgos

La gestión de riesgos abarca los procesos de planificación, identificación, análisis cualitativo/cuantitativo, planificación de respuesta y monitoreo continuo para reducir la probabilidad e impacto de eventos negativos.

Evaluación Cualitativa: Matriz de Probabilidad e Impacto

Permite priorizar los riesgos categorizándolos según su nivel de probabilidad de ocurrencia (P) y su impacto potencial en los objetivos del proyecto (I).
Nivel de Riesgo (Score) = Probabilidad × Impacto
Tabla de Valoración Sugerida:

Probabilidad / Impacto	Impacto Bajo (1)	Impacto Medio (3)	Impacto Alto (5)
Probabilidad Alta (5)	5 (Medio)	15 (Alto)	25 (Crítico)
Probabilidad Media (3)	3 (Bajo)	9 (Medio)	15 (Alto)
Probabilidad Baja (1)	1 (Bajo)	3 (Bajo)	5 (Medio)

Estrategias de Respuesta a Riesgos Negativos

- **Mitigar:** Reducir la probabilidad o el impacto (ej. realizar pruebas de carga antes del despliegue).
- **Evitar:** Cambiar el plan del proyecto para eliminar la amenaza por completo.
- **Transferir:** Trasladar el impacto a un tercero (ej. pólizas de seguro, contratos SLA con proveedores).
- **Aceptar:** Reconocer el riesgo sin tomar acción preventiva previa, contemplando reservas de contingencia.

Ética Profesional en Proyectos de TI

El ejercicio de la dirección de proyectos tecnológicos requiere no sólo rigor técnico, sino un sólido compromiso ético que proteja la privacidad, la seguridad de la información y la transparencia hacia el cliente y la sociedad.

Código de Ética y Conducta Profesional del PMI

Se estructura sobre cuatro valores fundamentales:

1. **Responsabilidad:** Tomar propiedad de las decisiones y sus consecuencias.
2. **Respeto:** Valorar a las personas, la diversidad y los recursos asignados.
3. **Equidad:** Actuar con imparcialidad, evitando conflictos de interés o favoritismos.
4. **Honestidad:** Proporcionar información veraz en las estimaciones, reportes de estado e indicadores financieros.

Dilemas Éticos Frecuentes en TI

- Estimaciones deliberadamente poco realistas para ganar una licitación.
- Uso impropio o no autorizado de datos personales (privacidad y cumplimiento normativo).
- Ocultamiento de fallos críticos de seguridad en el código para cumplir con la fecha de entrega acordada.

Bibliografía y Lecturas Complementarias

- **AXELOS.** (2017). *Managing Successful Projects with PRINCE2* (6th ed.). TSO.
- **AXELOS.** (2019). *ITIL Foundation: ITIL 4 Edition*. TSO.
- **Blank, L., & Tarquin, A.** (2012). *Ingeniería Económica* (7.ª ed.). McGraw-Hill.
- **ISACA.** (2018). *COBIT 2019 Framework: Governance and Management Objectives*. ISACA.
- **Project Management Institute (PMI).** (2021). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition and The Standard for Project Management*. PMI.
- **Project Management Institute (PMI).** (2020). *PMI Code of Ethics and Professional Conduct*. PMI.
- **Sapag, N., & Sapag, R.** (2017). *Preparación y evaluación de proyectos* (6.ª ed.). McGraw-Hill.